

1.2.1 描画フレームの流れ

1 描画フレーム = 動画の 1/24 秒 (`cfg::FRAME_DT`)。物理時間は実時間に依存せず固定ステップで進む。

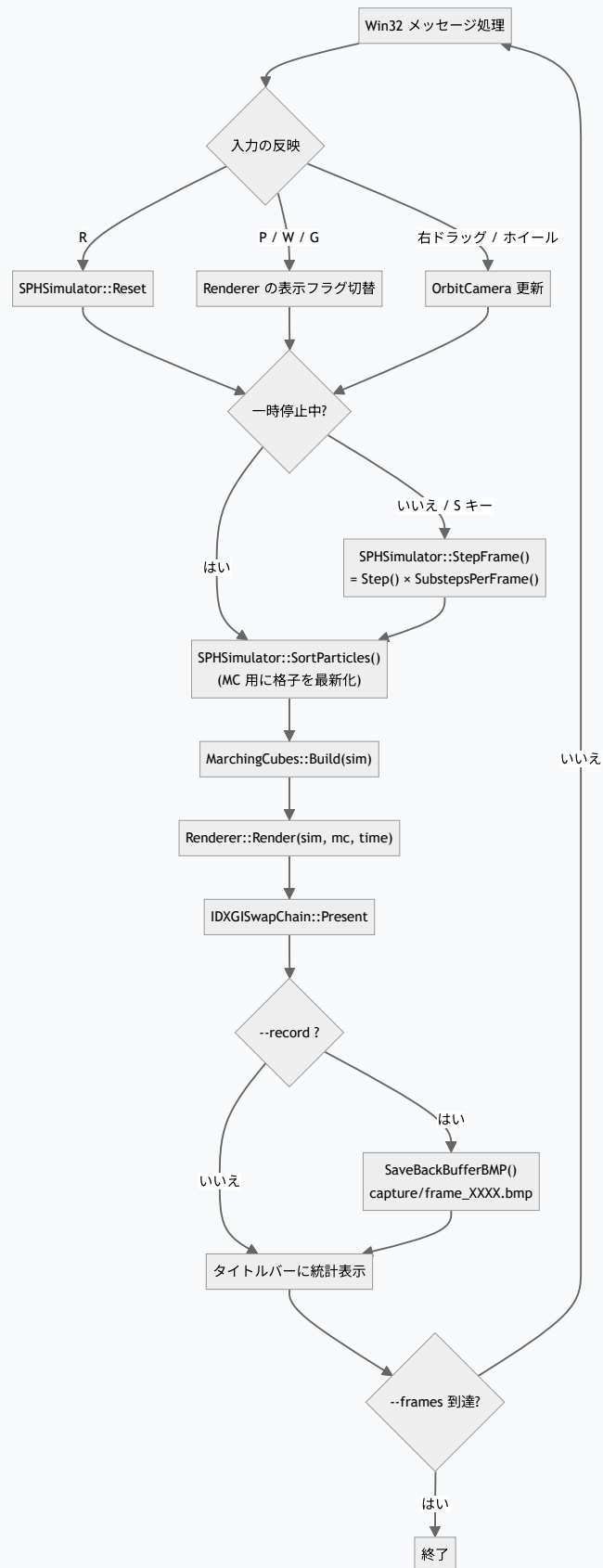


FIG 2 main.cpp のメインループ

2 コマンドラインとキー操作

2.1 コマンドライン (main.cpp)

オプション	引数	既定	説明
<code>--spacing</code>	粒子間隔 [m]	<code>0.014</code>	小さいほど粒子数が増え高精細 (粒子数 $\propto 1/\text{spacing}^3$)
<code>--record</code>	保存先ディレクトリ (省略可)	<code>capture</code>	各フレームを 24bit BMP で連番保存
<code>--frames</code>	フレーム数	無制限	指定フレーム描画後に自動終了
<code>--particles</code>	なし	-	粒子を点で表示するデバッグモードで開始
<code>--resttest</code>	なし	-	水塊 A を床に静置する安定性テストシーン
<code>--debug</code>	なし	-	D3D11 デバッグレイヤーを有効化

2.2 キー操作

キー	動作
<code>Space</code>	一時停止 / 再開
<code>S</code>	1 フレーム進める
<code>R</code>	初期状態へリセット
<code>P</code>	粒子ポイント表示の切替
<code>W</code>	水面メッシュ表示の切替
<code>G</code>	ガラス水槽表示の切替
右ドラッグ	カメラ回転 (ヨー / ピッチ)
ホイール	ズーム
<code>Esc</code>	終了

3 D3DUtil.h — DirectX 11 ユーティリティ

3.1 ThrowIfFailed(HRESULT hr, const char* what)

FAILED(hr) のとき `std::runtime_error("<what> failed (hr=0x....)")` を送出する。

3.2 struct GpuBuffer

メンバ	型	説明
buffer	ComPtr<ID3D11Buffer>	バッファ本体
srv	ComPtr<ID3D11ShaderResourceView>	読み取り用ビュー (t#)
uav	ComPtr<ID3D11UnorderedAccessView>	読み書き用ビュー (u#)
count	UINT	要素数
stride	UINT	要素サイズ [byte]

3.3 class D3DContext

メソッド	説明
<code>void Initialize(HWND hwnd, int width, int height, bool debugLayer)</code>	デバイス・スワップチェーン (R8G8B8A8, 2 バッファ) 生成。シェーダーディレクトリを <code><exe>/shaders/</code> → <code><exe>/../shaders/</code> → <code>shaders/</code> の順に探索
<code>Device()</code> / <code>Context()</code> / <code>SwapChain()</code>	生オブジェクトへのアクセサ
<code>BackBufferRTV()</code> / <code>BackBuffer()</code>	バックバッファ RTV / テクスチャ
<code>int Width()</code> / <code>int Height()</code>	バックバッファ解像度
<code>GpuBuffer CreateStructured(UINT stride, UINT count, const void* initData = nullptr, bool append = false)</code>	構造化バッファ + SRV/UAV。 <code>append=true</code> で Append カウンタ付き UAV
<code>GpuBuffer CreateRaw(UINT byteSize, bool indirectArgs = false)</code>	ByteAddressBuffer + RAW SRV/UAV。 <code>indirectArgs=true</code> で <code>DrawInstancedIndirect</code> 引数用
<code>ComPtr<ID3D11Buffer> CreateConstantBuffer(UINT byteSize)</code>	動的定数バッファ (16 byte 境界に切り上げ)
<code>ComPtr<ID3D11Buffer> CreateVertexBuffer(const void* data, UINT byteSize)</code>	静的 (IMMUTABLE) 頂点バッファ
<code>void UpdateConstantBuffer(ID3D11Buffer* cb, const void* data, UINT byteSize)</code>	<code>Map(WRITE_DISCARD)</code> で更新
<code>ComPtr<ID3DBlob> CompileShader(const std::wstring& file, const char* entry, const char* target)</code>	<code>D3DCompileFromFile</code> 。失敗時はエラー文を <code>stderr</code> に出して例外
<code>CreateCS / CreateVS / CreatePS(file, entry[, outBlob])</code>	<code>cs_5_0</code> / <code>vs_5_0</code> / <code>ps_5_0</code> でコンパイルして生成。 <code>CreateVS</code> は入力レイアウト用に blob を返せる
<code>void UnbindCompute()</code>	CS の t0-t7 / u0-u7 を全て解除 (SRV/UAV 競合防止)
<code>const std::wstring& ShaderDir()</code>	解決済みシェーダーディレクトリ

4 SPH Simulator — GPU WCSPH

4.1 struct SimParams (定数バッファ b0)

HLSL の cbuffer SimParams (sph_common.hlsl) と完全一致させる。

メンバ	型	意味
domainMin[3], cellSize	float	領域最小 / 格子セル幅 (= h)
domainMax[3], h	float	領域最大 / 平滑化長
gridDim[3], numCells	uint	格子分割数 / 総セル数
h2, mass	float	h^2 / 粒子質量 (格子上で ρ_0 になるよう自動決定)
restDensity, taitB	float	ρ_0 / Tait 剛性 $B = \rho_0 c^2 / \gamma$
viscosity, xsphEps	float	粘性 μ / XSPH 係数
dt, gravity	float	サブステップ時間 / 重力 (フルード相似で $1/SCENE_SCALE$)
poly6, spikyGrad, viscLap	float	カーネル正規化定数
tankInner, tankOuter, tankHeight, tankFloorY, floorY	float	水槽と床の形状
restitution, friction	float	壁の反発係数 / 接触中の接線減衰
maxSpeed, boundaryDensityScale	float	速度上限 / 境界密度補正の強さ
numParticles	uint	粒子数
artificialAlpha, cohesion, soundSpeed	float	Monaghan 人工粘性 α / 凝集力 κ / 数値音速 c

4.2 class SPH Simulator

メソッド	説明
<code>void Initialize(D3DContext& d3d, float spacing, bool restTest = false)</code>	パラメータ導出 ($h = 2 \cdot \text{spacing}$, $dt = \text{CFL} \cdot h/c$ からサブステップ数決定)、 <code>cfg::WATER_BLOCKS</code> から粒子生成、GPU バッファとシェーダー生成
<code>void Reset()</code>	初期位置・速度 0 を再アップロード、時刻 0
<code>void SortParticles()</code>	格子構築 (Clear → Count → Scan → Reorder) のみ実行。ピンポンバッファが入れ替わる
<code>void Step()</code>	1 サブステップ (SortParticles → Density → Force → Integrate)
<code>void StepFrame()</code>	<code>Step()</code> を <code>SubstepsPerFrame()</code> 回
<code>NumParticles()</code> / <code>SubstepsPerFrame()</code> / <code>SubstepDt()</code> / <code>SimTime()</code>	統計・状態
<code>const SimParams& Params()</code>	現在のパラメータ
<code>ID3D11Buffer* ParamsCB()</code>	定数バッファ (MC が <code>b0</code> に束縛して再利用)
<code>ID3D11ShaderResourceView* PositionSRV()</code>	現在の位置バッファ (<code>float4</code> × N、直近のソート順)
<code>ID3D11ShaderResourceView* CellStartSRV()</code>	セル先頭インデックス (<code>uint</code> × numCells+1)

前提条件: `PositionSRV()` と `CellStartSRV()` を近傍探索に使う側 (MC) は、直前に `SortParticles()` を呼び格子と位置の整合を取ること。 `Step()` 末尾の Integrate 後は位置が動いているため、そのままでは格子と一致しない。

4.3 サブステップのパイプライン (sph.hlsl)

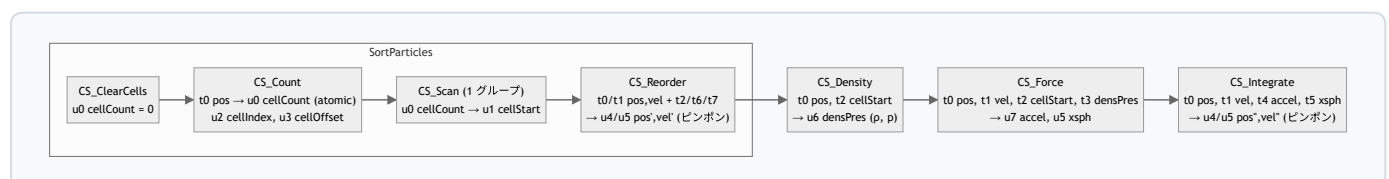


FIG 3 1 サブステップ = 7 回の Dispatch。各 Dispatch 後に UnbindCompute()

パス	内容
CS_Density	$\rho_i = m \sum W_{\text{poly6}} + \text{境界ゴースト密度}$ 、 $p = \max(0, B((\rho/\rho_0)^7 - 1))$
CS_Force	圧力項 $-\sum m (p_i/\rho_i^2 + p_j/\rho_j^2) \nabla W_{\text{spiky}}$ 、粘性項 $\mu \sum m (v_j - v_i)/\rho_j \nabla^2 W_{\text{visc}} / \rho_i$ 、Monaghan 人工粘性 (接近対のみ)、凝集力 $-\kappa \sum (x_i - x_j) W_{\text{poly6}}$ 、XSPH $\sum (m/\rho_j)(v_j - v_i) W_{\text{poly6}}$ 、重力
CS_Integrate	$v += a \cdot dt$ 、速度上限、 $x += (v + \varepsilon \cdot \text{xsph}) \cdot dt$ 、水槽壁 (高さ tankHeight 以下) / 床 / 領域 AABB との衝突

4.4 GPU バッファ一覧

バッファ	型	要素数	用途
m_pos[2], m_vel[2]	float4	N	位置・速度 (ピンポン)
m_xsph	float4	N	XSPH 補正速度
m_accel	float4	N	加速度
m_densPres	float2	N	密度・圧力
m_cellIndex, m_cellOffset	uint	N	粒子の所属セル / セル内順序
m_cellCount	uint	numCells	セル内粒子数
m_cellStart	uint	numCells + 1	排他的累積和 (末尾 = N)

5 MarchingCubes — GPU マーチングキューブ法

5.1 struct MCPParams (定数バッファ b1)

メンバ	型	意味
origin[3], cell	float	格子原点 / 格子間隔 (= spacing × 0.75)
dim[3], numVerts	uint	格子点数 (各軸) / 総格子点数
isoLevel, fieldRadius, fieldInvR2	float	等値面レベル / カーネル半径 $R (= h) / 1/R^2$
tableStride, maxTriangles	uint	テーブルの 1 パターン長 (32) / 三角形バッファ容量

5.2 class MarchingCubes

メソッド	説明
<code>void Initialize(D3DContext& d3d, float spacing)</code>	格子設定、 <code>mc::GenerateTriangleTable()</code> + <code>mc::SelfTest()</code> (失敗時は例外)、GPU リソース生成
<code>void Build(SPHSimulator& sim)</code>	現在の粒子位置からメッシュを生成 (§ 5.3)。呼び出し前に <code>sim.SortParticles()</code> 済みであること
<code>ID3D11ShaderResourceView* TriangleSRV()</code>	出力三角形 (<code>MCTriangle</code> × <code>maxTriangles</code>)
<code>ID3D11Buffer* DrawArgs()</code>	<code>DrawInstancedIndirect</code> 引数 {頂点数, 1, 0, 0}
<code>const MCPParams& Params()</code>	格子パラメータ
<code>uint32_t LastTriangleCount()</code>	三角形数を CPU に読み戻す (GPU 同期を伴うので統計表示用)

5.3 メッシュ生成のパイプライン (mc.hlsl)

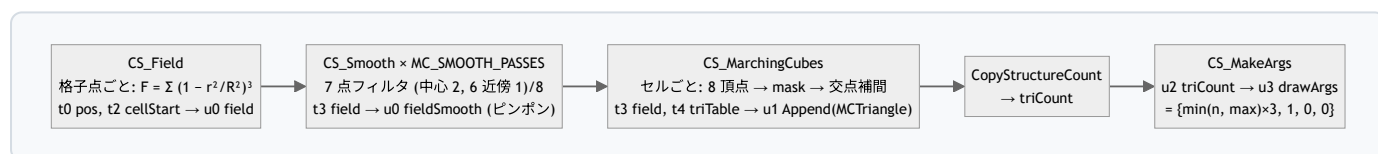


FIG 4 MarchingCubes::Build() — CPU への読み戻し無しで描画引数まで GPU 内で完結

出力頂点フォーマット (`water.hlsl` と共有):

```
struct MCVertex    { float3 p; float3 n; };           // 位置, 法線 (= -∇F を正規化)
struct MCTriangle { MCVertex v0, v1, v2; };           // 72 byte、頂点順は外向き
```

5.4 namespace mc (MCTables.h)

要素	説明
<code>NUM_CASES = 256</code> , <code>TABLE_STRIDE = 32</code>	テーブル寸法
<code>EDGE_VERTS[12][2]</code>	エッジ両端の頂点番号。頂点 v の座標 = $(v \& 1, (v \gg 1) \& 1, (v \gg 2) \& 1)$ 、エッジ $0-3 = x$ 、 $4-7 = y$ 、 $8-11 = z$ 方向
<code>std::vector<int32_t></code> <code>GenerateTriangleTable()</code>	256 パターンの三角形テーブルを生成 (エッジ番号 3 個ずつ、-1 終端)。曖昧面は「内側頂点を分離」規則で統一、ループの向きは外向き
<code>bool SelfTest(const std::vector<int32_t>& table, int32_t* outTriangleCount = nullptr)</code>	球の距離場で CPU MC を実行し、閉多様体 (各有向エッジが 1 回、逆向きも 1 回) と外向きを検証

6 Renderer — 描画

6.1 struct FrameParams (定数バッファ b0、VS/PS 共通)

メンバ	意味
<code>viewProj</code> , <code>view</code> , <code>lightViewProj</code>	行列 (転置済み、HLSL は <code>mul(vec, M)</code>)
<code>cameraPos</code>	xyz = カメラ位置
<code>lightDir</code>	xyz = 光源へ向かう単位ベクトル
<code>screenSize</code>	xy = 内部解像度, zw = 逆数
<code>params</code>	x = 時刻, y = near, z = far
<code>tank</code>	x = 内側半幅, y = 外側半幅, z = 描画するガラス高さ, w = 床の高さ

6.2 struct OrbitCamera

メンバ / メソッド	説明
<code>target</code> , <code>yawDeg</code> , <code>pitchDeg</code> , <code>distance</code> , <code>fovDeg</code>	注視点まわりの軌道カメラ
<code>XMVECTOR Eye()</code>	視点位置
<code>XMMATRIX View()</code>	<code>XMMatrixLookAtLH</code>
<code>XMMATRIX Proj(float aspect, float nearZ, float farZ)</code>	<code>XMMatrixPerspectiveFovLH</code>

6.3 class Renderer

メソッド / メンバ	説明
<code>void Initialize(D3DContext& d3d)</code>	レンダーターゲット (内部解像度 = ウィンドウ × SSAA)、メッシュ、ステート、シェーダー生成
<code>void Render(SPHSimulator& sim, MarchingCubes& mc, float time)</code>	§ 6.4 の全パスを実行してバックバッファへ出力 (Present は呼ばない)
<code>OrbitCamera& Camera()</code>	カメラ (外部から編集可)
<code>bool showParticles / showWater / showGlass</code>	表示フラグ

6.4 描画パスの流れ

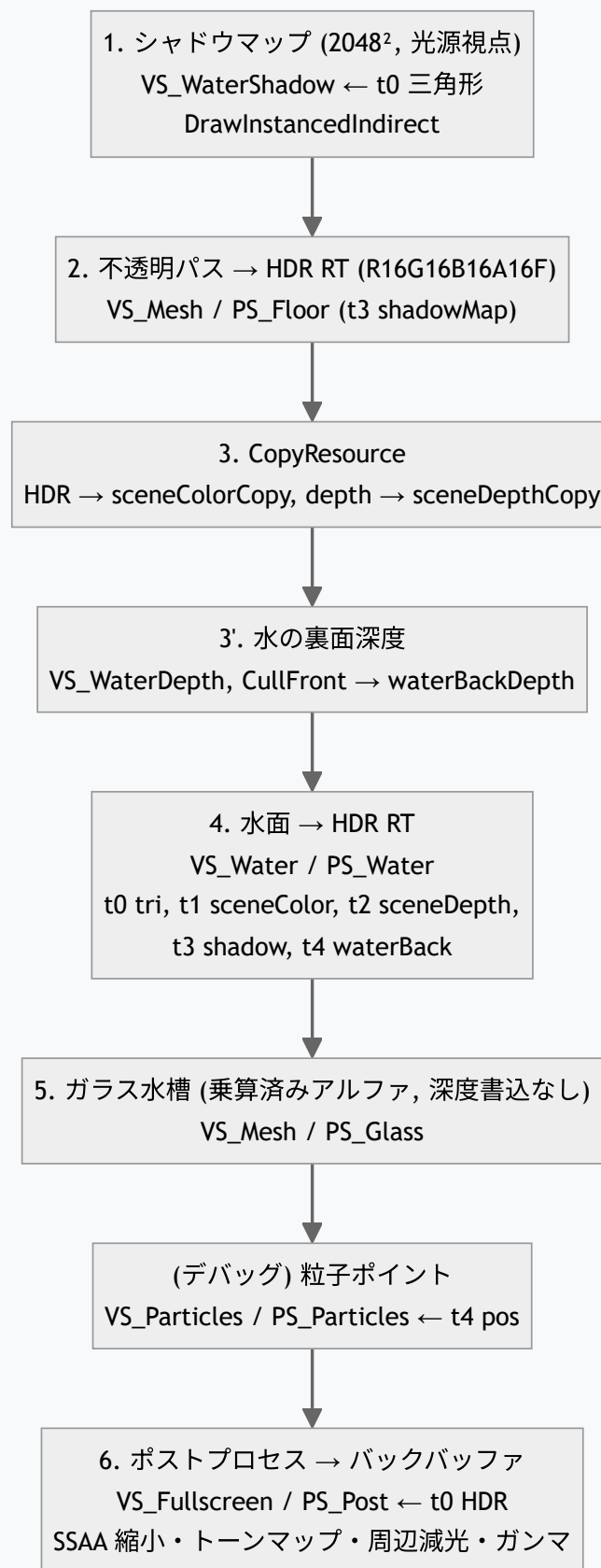


FIG 5 Renderer::Render() — 番号付きは常時実行、(デバッグ) は showParticles 時のみ

PS_Water の色: `lerp`(屈折, 反射, フレネル) + ハイライト × 影。屈折 = `sceneColorCopy` を法線ですらして参照 × `exp`(-厚み × `absorb`)、厚み = `min`(背景深度, 裏面深度) - 表面深度。

7 HLSL リソース束縛一覧

7.1 定数バッファ

スロット	構造体	使用シェーダー
b0	SimParams	sph.hlsl, mc.hlsl (格子の再利用)
b1	MCPParams	mc.hlsl
b0	FrameParams	water.hlsl, scene.hlsl, post.hlsl

7.2 sph.hlsl

種別	スロット	名前	型
SRV	t0 / t1	gPosIn / gVelIn	StructuredBuffer<float4>
SRV	t2	gCellStart	StructuredBuffer<uint>
SRV	t3	gDensPres	StructuredBuffer<float2>
SRV	t4 / t5	gAcce1 / gXsph	StructuredBuffer<float4>
SRV	t6 / t7	gCellIndex / gCellOffset	StructuredBuffer<uint>
UAV	u0 / u1	gCellCount / gCellStartRW	RWStructuredBuffer<uint>
UAV	u2 / u3	gCellIndexRW / gCellOffsetRW	RWStructuredBuffer<uint>
UAV	u4 / u5	gPosOut / gVelOut (CS_Force では XSPH 出力)	RWStructuredBuffer<float4>
UAV	u6	gDensPresRW	RWStructuredBuffer<float2>
UAV	u7	gAcce1RW	RWStructuredBuffer<float4>

エントリポイント: `CS_ClearCells`, `CS_Count`, `CS_Scan` (1024 スレッド × 1 グループ), `CS_Reorder`, `CS_Density`, `CS_Force`, `CS_Integrate` (256 スレッド/グループ)

7.3 mc.hlsl

種別	スロット	名前	型
SRV	t0	gPos	StructuredBuffer<float4>
SRV	t2	gCellStart	StructuredBuffer<uint>
SRV	t3	gField	StructuredBuffer<float>
SRV	t4	gTriTable	StructuredBuffer<int>
UAV	u0	gFieldRW	RWStructuredBuffer<float>
UAV	u1	gTriangles	AppendStructuredBuffer<MCTriangle>
UAV	u2 / u3	gTriCount / gDrawArgs	RWByteAddressBuffer

エントリポイント: CS_Field , CS_Smooth , CS_MarchingCubes , CS_MakeArgs

7.4 描画シェーダー

ファイル	エントリ	入力	主なリソース
water.hlsl	VS_Water , VS_WaterShadow , VS_WaterDepth	SV_VertexID (頂点バ ッファ無し)	t0 gTriangles
water.hlsl	PS_Water	-	t1 gSceneColor, t2 gSceneDepth, t3 gShadowMap, t4 gWaterBack
water.hlsl	VS_Particles / PS_Particles	SV_VertexID (粒子あ たり 6 頂点)	t4 gParticlePos
scene.hlsl	VS_Mesh	POSITION(float3), NORMAL(float3)	-
scene.hlsl	PS_Floor / PS_Glass	-	t3 gShadowMap
post.hlsl	VS_Fullscreen / PS_Post	SV_VertexID (3 頂点)	t0 gHdr
共通	-	-	s0 gLinearClamp, s1 gShadowCmp (比較サンプリ)

8 初期化シーケンス

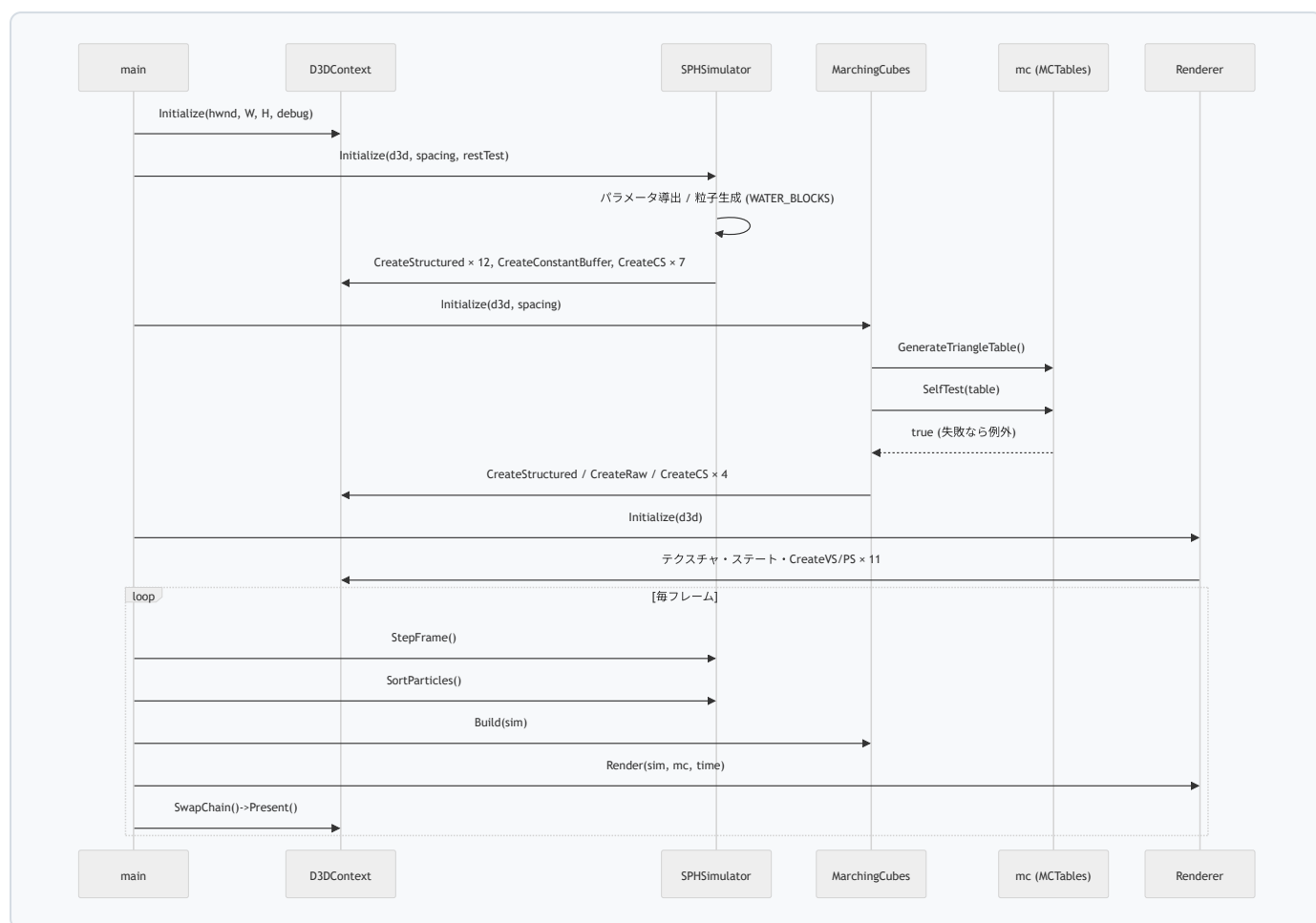


FIG 6 起動から定常ループまで

9 エラーと制約

状況	挙動
シェーダーコンパイル失敗	<code>D3DContext::CompileShader</code> が <code>stderr</code> にエラー文を出力し <code>std::runtime_error</code> を送出。 <code>main</code> が捕捉して <code>MessageBox</code> 表示後に 終了コード 1
MC テーブル自己検証失敗	<code>MarchingCubes::Initialize</code> が例外
三角形数が <code>MC_MAX_TRIANGLES</code> (1,048,576) を超過	超過分の <code>Append</code> は破棄され、 <code>CS_MakeArgs</code> が描画数を容量でクラン プ (穴が空くが落ちない)
粒子の速度超過	<code>maxSpeed = 0.5 h / dt</code> でクランプ (爆発防止)
SRV/UAV の同時束縛	各 <code>Dispatch</code> 後に <code>UnbindCompute()</code> で解除する規約
機能レベル	D3D 11.0 以上 (UAV は u0-u7 の範囲のみ使用)

10 主要パラメータの既定値 (SimConfig.h)

定数	値	意味
WINDOW_W × WINDOW_H , SSAA	1280 × 720, 2	出力解像度と内部倍率
VIDEO_FPS , FRAME_DT	24, 1/24	固定ステップ
DEFAULT_SPACING , H_PER_SPACING	0.014, 2.0	粒子間隔, $h = 2d$
REST_DENSITY , SOUND_SPEED , TAIT_GAMMA	1000, 12, 7	状態方程式
VISCOSITY_MU , ARTIFICIAL_VISC_ALPHA , XSPH_EPS	0.5, 0.08, 0.15	減衰項
SCENE_SCALE , GRAVITY	2.0, 9.81/2	フルード相似
CFL	0.4	$dt = CFL \cdot h/c$
TANK_INNER_HALF , TANK_HEIGHT , GLASS_VISIBLE_HEIGHT	0.5, 1.6, 0.3	水槽 (衝突は 1.6 まで、描画は 0.3 まで)
MC_CELL_PER_SPACING , MC_FIELD_RADIUS_PER_SPACING , MC_ISO_LEVEL , MC_SMOOTH_PASSES	0.75, 2.0, 0.5, 2	メッシュ生成
CAM_YAW_DEG , CAM_PITCH_DEG , CAM_DISTANCE , CAM_FOV_DEG	-45, 30, 3.2, 40	カメラ
LIGHT_DIR	(0.15, 1, -0.9)	光源方向